

Python pathlib Module

```
# =====
# PATHLIB MODULE - COMPLETE EXPLANATION IN ONE SCRIPT
# =====

from pathlib import Path

# =====
# 1. CREATE A PATH OBJECT
# =====

file_path = Path("data/customers.csv")

print("Path:", file_path)
print("Type:", type(file_path))

# =====
# 2. CURRENT WORKING DIRECTORY
# =====

current_directory = Path.cwd()

print("\nCurrent working directory:")
print(current_directory)

# =====
# 3. HOME DIRECTORY
# =====

home_directory = Path.home()

print("\nHome directory:")
print(home_directory)

# =====
# 4. CHECK IF PATH EXISTS
# =====

path = Path("data")

print("\nDoes path exist?")
print(path.exists())

# =====
# 5. CHECK FILE OR DIRECTORY
# =====

file_path = Path("customers.csv")
directory_path = Path("data")

print("\nIs file?")
print(file_path.is_file())

print("\nIs directory?")
print(directory_path.is_dir())
```

```

# =====
# 6. CREATE A DIRECTORY
# =====

data_directory = Path("data")

data_directory.mkdir(
    exist_ok=True
)

print("\nData directory created.")

# =====
# 7. CREATE NESTED DIRECTORIES
# =====

output_directory = Path("data/output/processed")

output_directory.mkdir(
    parents=True,
    exist_ok=True
)

print("Nested directories created.")

# =====
# 8. BUILD PATHS
# =====

data_directory = Path("data")

csv_file = data_directory / "customers.csv"

print("\nCSV file path:")
print(csv_file)

# Another example

output_directory = Path("data") / "output"

json_file = output_directory / "customers.json"

print("JSON file path:")
print(json_file)

# =====
# 9. FILE NAME
# =====

file_path = Path("data/customers.csv")

print("\nFile name:")
print(file_path.name)

# =====
# 10. FILE STEM
# =====

print("\nFile stem:")
print(file_path.stem)

# customers.csv
#   ↓
# customers

```

```

# =====
# 11. FILE SUFFIX / EXTENSION
# =====

print("\nFile extension:")
print(file_path.suffix)

# .csv

# =====
# 12. PARENT DIRECTORY
# =====

print("\nParent directory:")
print(file_path.parent)

# =====
# 13. MULTIPLE SUFFIXES
# =====

file_path = Path("backup/customers.csv.gz")

print("\nSuffix:")
print(file_path.suffix)

print("All suffixes:")
print(file_path.suffixes)

# =====
# 14. WRITE TEXT TO A FILE
# =====

file_path = Path("data/example.txt")

file_path.write_text(
    "Hello from Python!\n"
    "This file was created using pathlib."
)

print("\nFile written.")

# =====
# 15. READ TEXT FROM A FILE
# =====

content = file_path.read_text()

print("\nFile content:")
print(content)

# =====
# 16. CHECK FILE SIZE
# =====

file_size = file_path.stat().st_size

print("\nFile size:")
print(file_size, "bytes")

```

```

# =====
# 17. LIST FILES IN A DIRECTORY
# =====

data_directory = Path("data")

print("\nFiles and directories inside data:")

for item in data_directory.iterdir():

    print(item)

# =====
# 18. FIND CSV FILES
# =====

print("\nCSV files:")

for file in data_directory.glob("*.csv"):

    print(file)

# =====
# 19. FIND JSON FILES
# =====

print("\nJSON files:")

for file in data_directory.glob("*.json"):

    print(file)

# =====
# 20. RECURSIVE SEARCH
# =====

print("\nAll CSV files recursively:")

for file in data_directory.rglob("*.csv"):

    print(file)

# =====
# 21. RENAME A FILE
# =====

old_file = Path("data/example.txt")
new_file = Path("data/customers.txt")

if old_file.exists():

    old_file.rename(new_file)

    print("\nFile renamed.")

# =====
# 22. DELETE A FILE
# =====

temporary_file = Path("data/temp.txt")

temporary_file.write_text("Temporary data")

print("\nTemporary file created.")

```

```

temporary_file.unlink()

print("Temporary file deleted.")

# =====
# 23. READ FILE LINE BY LINE
# =====

file_path = Path("data/customers.txt")

if file_path.exists():
    print("\nReading lines:")
    for line in file_path.read_text().splitlines():
        print(line)

# =====
# 24. PATH AS STRING
# =====

file_path = Path("data/customers.csv")

path_string = str(file_path)

print("\nPath as string:")
print(path_string)

print(type(path_string))

# =====
# 25. ABSOLUTE PATH
# =====

file_path = Path("data/customers.csv")

print("\nAbsolute path:")
print(file_path.absolute())

# =====
# 26. RESOLVE PATH
# =====

print("\nResolved path:")
print(file_path.resolve())

# =====
# 27. PATH PARTS
# =====

file_path = Path("data/raw/customers.csv")

print("\nPath parts:")

for part in file_path.parts:
    print(part)

```

```

# =====
# 28. CHANGE FILE EXTENSION
# =====

file_path = Path("data/customers.csv")

json_path = file_path.with_suffix(".json")

print("\nOriginal:")
print(file_path)

print("New extension:")
print(json_path)

# =====
# 29. CHANGE FILE NAME
# =====

file_path = Path("data/customers.csv")

new_path = file_path.with_name("customers_clean.csv")

print("\nOriginal:")
print(file_path)

print("New name:")
print(new_path)

# =====
# 30. DATA ENGINEERING EXAMPLE
# =====

# Imagine an ETL project:

project_directory = Path("etl_project")

raw_directory = project_directory / "raw"
processed_directory = project_directory / "processed"
output_directory = project_directory / "output"

# Create directories

raw_directory.mkdir(
    parents=True,
    exist_ok=True
)

processed_directory.mkdir(
    parents=True,
    exist_ok=True
)

output_directory.mkdir(
    parents=True,
    exist_ok=True
)

print("\n===== ETL DIRECTORIES =====")

print("Raw:", raw_directory)
print("Processed:", processed_directory)
print("Output:", output_directory)

```

```

# =====
# 31. FIND RAW DATA FILES
# =====

# Suppose raw files exist here:

raw_files = [
    raw_directory / "customers.csv",
    raw_directory / "orders.csv",
    raw_directory / "products.csv"
]

print("\nRaw files:")

for file in raw_files:
    print(file)

# =====
# 32. BUILD PROCESSED FILE PATHS
# =====

for raw_file in raw_files:
    processed_file = (
        processed_directory /
        f"{raw_file.stem}_clean{raw_file.suffix}"
    )

    print(
        "\nRaw:",
        raw_file
    )

    print(
        "Processed:",
        processed_file
    )

# =====
# 33. DATA PIPELINE DIRECTORY STRUCTURE
# =====

pipeline = Path("data_pipeline")

directories = [
    pipeline / "raw",
    pipeline / "staging",
    pipeline / "processed",
    pipeline / "archive",
    pipeline / "logs"
]

for directory in directories:
    directory.mkdir(
        parents=True,
        exist_ok=True
    )

print("\nPipeline structure created:")

for directory in pipeline.rglob("*"):
    if directory.is_dir():
        print(directory)

```

```

# =====
# 34. PROCESS ALL CSV FILES
# =====

raw_directory = pipeline / "raw"

csv_files = list(
    raw_directory.glob("*.csv")
)

print("\nCSV files to process:")

for csv_file in csv_files:
    print(csv_file)

# =====
# 35. PATHLIB + JSON ETL
# =====

import json

output_directory = Path("data/json_output")

output_directory.mkdir(
    parents=True,
    exist_ok=True
)

customer_data = {
    "id": 1,
    "name": "Ahmed",
    "city": "Cairo"
}

output_file = output_directory / "customer.json"

with output_file.open("w") as file:
    json.dump(
        customer_data,
        file,
        indent=4
    )

print("\nJSON file created:")
print(output_file)

# =====
# 36. PATHLIB + CSV ETL
# =====

import csv

csv_directory = Path("data/csv_output")

csv_directory.mkdir(
    parents=True,
    exist_ok=True
)

csv_file = csv_directory / "customers.csv"

customers = [
    ["id", "name", "city"],
    [1, "Ahmed", "Cairo"],
    [2, "Mohamed", "Giza"],

```

```

        [3, "Omar", "Alexandria"]
    ]

    with csv_file.open(
        "w",
        newline=""
    ) as file:

        writer = csv.writer(file)

        writer.writerows(customers)

    print("\nCSV file created:")
    print(csv_file)

```

```

# =====
# 37. IMPORTANT PATHLIB METHODS
# =====

```

```

print("""
=====
PATHLIB - IMPORTANT METHODS
=====

```

```

Path.cwd()
    Current working directory

Path.home()
    User home directory

Path.exists()
    Check if path exists

Path.is_file()
    Check if path is a file

Path.is_dir()
    Check if path is a directory

Path.mkdir()
    Create directory

Path.iterdir()
    List directory contents

Path.glob()
    Find files using pattern

Path.rglob()
    Recursive file search

Path.read_text()
    Read text file

Path.write_text()
    Write text file

Path.rename()
    Rename/move file

Path.unlink()
    Delete file

Path.stat()
    File information

Path.absolute()
    Absolute path

```

`Path.resolve()`
Resolved absolute path

`Path.with_suffix()`
Change extension

`Path.with_name()`
Change filename

=====

IMPORTANT PROPERTIES

=====

`path.name`
Filename

`path.stem`
Filename without extension

`path.suffix`
Extension

`path.suffixes`
All extensions

`path.parent`
Parent directory

`path.parts`
Path components

=====

MOST IMPORTANT CONCEPT

=====

Instead of:

`"data/output/customers.csv"`

You can use:

`Path("data") / "output" / "customers.csv"`

This is cleaner and more portable across
Windows, Linux, and macOS.

=====

DATA ENGINEERING

=====

RAW DATA

↓

`Path("data/raw")`

↓

Read files

↓

Transform

↓

`Path("data/processed")`

↓

Save files

↓

`Path("data/output")`

↓
Final Data
=====

""")